

Machine Learning Engineer, Practical Exercise

Senior ML Engineer role · Take-home practical

Time: we expect a strong submission to take around **3 focused hours**, treat that as a *budget, not a rule*. How you balance scope against time is part of what we're assessing, so build what you'd confidently ship in that window and tell us what you traded off. **Deadline:** within 48 hours of receipt (schedule the time whenever suits you).

Submit: a GitHub repo link + a short video link, in a single reply.

AI tools: allowed, and we assume you'll use them. We only require that you can explain and defend every decision you ship.

The setting

You're taking over support automation for a crypto/fintech product. Two things are waiting for you: a **route classifier** a previous engineer wants to ship, and a **knowledge base** that customer questions need answering from. The starter pack has both.

The KB is a real one in the way that matters: **policies change over time**. Fees get repriced, procedures get replaced, notices expire. Every document carries `effective_date`, `valid_until`, `status` and `supersedes` metadata. Answering from the wrong version of a document is the failure mode this exercise cares most about.

Part A, review the baseline classifier required

`baseline/baseline_classifier.py` and `baseline/eval_report.md` were handed to you by the engineer who wrote them. They report **98.75% accuracy** and recommend shipping.

Do you sign off? Write your answer in the README. Whatever you conclude, we want:

- What you'd change about **how it is evaluated**, and what the honest number is.
- The metric you'd hold this service to in production, given that **fraud-report** is the most expensive route to get wrong.
- A working version of your fix. A small diff on their file is completely fine, this part should not eat your budget.

Part B, answering from the knowledge base required

Build a service that takes a question and returns an answer grounded in `kb/`.

- An **answer interface**, `answer(question, as_of) → answer + the documents used`.
- A **batch entry point** that reads `questions.csv` and writes `answers.csv` with `qid`, `answer`, `doc_ids`.
- **Correct as at a date**. Each question carries an `as_of`. Some are historical. An answer that is right today can be wrong for the date being asked about.
- **An honest evaluation of your retrieval**. Not just "it looks right", tell us how you measured it and what it scores.
- **Not every question is answerable from the KB**. Decide what your system does then, and defend it.
- **Clean, tested Python**: at least two meaningful tests and basic input validation.

An LLM is allowed but **not required**, and this is not a test of whether you can call one. If you use one, bring your own key and keep the repo runnable without it (a documented fallback, a cached fixture, or a clear skip). If you'd

rather do this with embeddings and no LLM, that is a legitimate answer and we will read it as one.

Part C, a second modality optional

Stretch item, not an expectation. `media/` holds 3 screenshots and 3 voice notes, all synthetic. **Pick one modality, not both.** Route those assets into the same four routes.

What we actually read here is the reasoning, not the API call:

- Why that modality, and why the approach you chose (OCR versus a vision model, local transcription versus hosted).
- What it costs per ticket and what it adds in latency.
- What happens when it degrades. A blurred screenshot, a heavy accent, a transcription that drops a word.
- These assets contain things a customer would not want leaked. What does that imply for where they get processed?

Deciding *not* to do Part C, and telling us why, costs you nothing.

A short "Scope & trade-offs" note

In your README *or* in the video, briefly tell us:

- What you **prioritized**, and what you **deliberately left out**.
- What you'd do with more time.
- Roughly how long you actually spent (honest estimate is fine).

Deliver via GitHub

- A repo (public, or invite us) with **real commit history**, please commit as you work, not in one final push.
- A **README** with clear run instructions and the exact commands for both entry points.

Plus a short video (≤ 8 min)

Voice required; screen-share optional. Walk us through your solution and answer these out loud:

1. You saw a baseline reporting 98.75%. What did you conclude, and what number would you defend to a product manager instead?
2. How does your system decide **which version** of a document to trust, and how would you find out in production that it had started serving stale answers?
3. Point to **one decision in your own code** you were unsure about, and explain the trade-off you made.
4. At **10,000 requests/minute**, which parts of your system need an LLM and which don't? Where would you cache, and what breaks when the underlying document changes?
5. *Only if you did Part C:* what fails first in that pipeline once it is live, and how would you catch it?

How we evaluate

- **ML mechanics & metric reasoning**, do you understand *why*, not just *what* (metric choice, imbalance, data leakage).
- **Retrieval & freshness**, does the right document win, do you know when it doesn't, and does your system know what it doesn't know.

- **Judgment**, choosing the right tool for the problem, not maximum complexity.
- **Prioritization & scoping**, sensible time-vs-features trade-offs, clearly reasoned.
- **Code quality & testing**, readable, structured, genuinely tested.
- **Communication**, clear, confident explanation of your decisions.

To submit: reply with (1) your GitHub repo link and (2) your video link. That's it. Thanks, we're looking forward to seeing how you think.

Top candidates from this exercise are invited to a 45-minute live session, which builds directly on the work you submit here.